# Native fork join: work stealing by context switching

Mike Rainey
Carnegie Mellon University
Pittsburgh, PA, USA
me@mike-rainey.site

## Abstract

TaskPaRTS is a C++ library for fork-join parallelism that requires no compiler support. At its core is a technique we call *native fork join*, which delivers work-stealing scheduling by context switching. When nothing is stolen, a fork is a push, two ordinary calls, and a pop: both branches run on the parent's stack, and the fork neither allocates memory nor switches contexts. When the join finds that the second branch was stolen, the worker neither blocks nor runs other work on top of the parent's frame. It saves the continuation of the parent as a handful of machine registers that point into the parent's own stack, leaves that stack suspended, and returns to its scheduler. Whichever worker finishes the last branch then resumes the parent, possibly on a different thread. In this short paper, we present the algorithm in pseudocode, explain why it is safe, and describe what it means for programs that use it. A companion implementation, micro taskparts, implements the same algorithm in a few hundred lines of C++.

## 1 Challenge: the join, in a library

The problem is to implement binary fork join: a function that runs two thunks, possibly in parallel, and returns once both have finished.

```
fork2join(f1, f2 : () → void) → void
```

For example:

```
fib(n : int) → int {
  if (n < 2) return n
  fork2join(λ () ⇒ {a = fib(n-1)},
            λ () ⇒ {b = fib(n-2)})
  return a + b }
```

Work stealing [4, 9] is well suited to scheduling such programs. Our constraint is that we are writing a library: there is no compiler support, and client code is ordinary C++, with lambdas for thunks.

A library work stealer typically implements `fork2join` by *child stealing*: the second branch becomes a task that thieves may take.

```
fork2join(f1, f2 : () → void) → void {
  t = new_task(f2); push(t)   // t is now stealable
  f1()
  if (pop() == t) { f2(); return }   // not stolen
  while (!t.done)                    // stolen: wait for thief
    { u = steal(); if (u != null) u.run() } }
```

The common case, where `t` is not stolen, is cheap. The difficulty is the join when `t` was stolen. The parent's frame sits on the worker's thread stack, and must stay there until the join completes. So the worker can either block, and sit idle, or, as TBB and ParlayLib do [3, 10], *help*: run other tasks on top of the blocked frame, as in the loop above. Helping keeps the worker busy, but the parent can resume only once the worker has unwound back down to its frame, even if the thief finished long before. Cilk [7, 17] avoids waiting by stealing the parent's continuation instead of the child. But then the thief must resume a frame that sits on the victim's stack, which in Cilk relies on compiler support and a cactus stack [11].

Native fork join, which is at the core of TaskPaRTS [14], takes a different route. When a worker finds at a join that the second branch was stolen, it *suspends the parent's stack* and returns to its scheduler, which looks for other work. The parent's continuation is a saved register context that points into the suspended stack. Whichever worker finishes the last branch restores this context, and so the parent continues on that worker's thread. No worker waits at a join, and no two workers ever run on the same stack at once.

We present the algorithm in Section 2, trace its two cases in Section 3, explain why it is safe in Section 4, and discuss its costs and its consequences for client code in Section 5. The appendices give machine-level details, the requirements on the work-stealing deque, the earlier formulation of native fork join that TaskPaRTS implements, and notes on performance. The last appendix compares native fork join with other ways of managing stacks under work stealing. Micro taskparts, a minimal C++ implementation of the algorithm, accompanies this paper.

## 2 Native fork join

***Contexts.*** Our one low-level primitive is the *context*. A context holds what the C calling convention requires a function call to preserve: the callee-saved registers, the stack pointer, and a return address. That is 8 words on x86-64 and 21 on AArch64 (Appendix A). `swap(`$k_1$`, `$k_2$`)` saves its caller's context in $k_1$ and resumes $k_2$; the call returns when some worker later resumes $k_1$. `restore(`$k$`)` resumes $k$ without saving anything, and so never returns. The worker that resumes a context need not be the one that saved it, and the context may point into any stack. To the compiler, `swap` is an ordinary call, so it spills any live caller-saved registers before the

```
type join = {pending : atomic int, k : context}
type task = {body : () → void, j : join*,
  k : context}
type worker = {k : context, dq : deque of task*,
  starting : task*, reached : join*}

loop() → void {   // on the worker's own stack
  while (!done) {
    j = me.reached; me.reached = null
    if (j != null) {   // a branch reached j
      if (fetch_and_add(j.pending, -1) == 1)
        swap(me.k, j.k)   // resume the parent
      continue }
    t = steal()
    if (t == null) continue
    t.k = new_context(new_stack(), enter)
    me.starting = t
    swap(me.k, t.k) } }

enter() → void {   // first frame on a fresh stack
  t = me.starting; t.body(); finish(t.j) }

finish(j : join*) → void {   // leave this stack
  if (j == null) done = true   // root
  me.reached = j; restore(me.k) }
```

**Figure 1.** The scheduler

call. As a result, a context switch is a handful of loads and stores, with no system call.

***Joins, tasks, and workers.*** Each call to `fork2join` keeps two records in its own frame (Figure 1). The *join* holds a count, `pending`, of the branches that have yet to reach the join, and a context, `k`, in which the parent is saved if it must be suspended. The *task* stands for the second branch, and is what thieves steal. It points to its join, and holds the context in which a thief starts it.

Each worker runs a scheduler loop on its thread's own stack, and client code on separate *task stacks*. While the worker runs client code, it keeps the loop's context in `k`. Each worker also has a work-stealing deque of tasks (Appendix B): `push(`$t$`)` adds $t$ to the caller's deque, `pop()` takes back the task pushed most recently, if it is still there, and `steal()` takes the oldest task of another worker, chosen at random. We write `me` for the worker that runs the caller, which we look up again at each use, because the worker can change across a call to `fork2join` (Section 5).

***Fork join.*** `fork2join` (Figure 2) has four steps.

(1) Push a task for the second branch, which makes it stealable, and run $f_1$ as an ordinary call.
(2) Pop.

```
fork2join(f1, f2 : () → void) → void {
  j = join{pending = 2}
  t = task{body = f2, j = &j}
  push(&t)                                  // (1)
  f1()
  if (pop() != null)                        // (2)
    { f2(); return }                        // (3)
  me.reached = &j                           // (4)
  swap(j.k, me.k) }
```

**Figure 2.** Fork join

(3) If the pop succeeds, it returns the task pushed at step 1, so the second branch was not stolen. Run $f_2$ as an ordinary call, and return.
(4) Otherwise, the second branch was stolen. Tell the scheduler loop that a branch has reached the join, and switch to the loop, saving the parent's context in the join. The call to `swap` returns, and with it `fork2join`, when both branches have reached the join.

***The scheduler.*** The scheduler loop (Figure 1) does two things. First, it counts the branches that reach a join. A branch never decrements the count of its join itself. Instead, it leaves its join in `reached` and switches to the loop, which decrements the count on its behalf. The worker whose decrement brings the count to zero resumes the parent at once, by switching to the context saved in the join. Second, when no branch has just reached a join, the loop steals. It gives a stolen task a fresh stack and a context that, once resumed, calls `enter` at the top of that stack. `enter` runs the body of the task, and then reports to the task's join, through the loop of whichever worker it is on by then.

A worker's deque is empty whenever the worker is in its loop, which is why the loop never pops. A worker enters its loop either from step 4, where the pop has just failed, or from `finish`, at the bottom of a task stack, where every task pushed on that stack has been popped or stolen.

A call to `fork2join` from outside the scheduler starts the workers, one of which starts a root task that makes the call. The root has no join, and the computation ends when it finishes.

## 3 The two cases

***Not stolen.*** Steps 1 to 3 are all that happens. $f_1$ and $f_2$ both run on the parent's stack, as ordinary calls, on one worker, and the join is never used. The fork costs a push and a pop.

***Stolen.*** Figure 3 traces the other case. Worker B steals the task from A while A runs $f_1$ on the parent's stack, S. B gives the task a fresh stack, T, and runs $f_2$ there. When the pop at step 2 fails, A suspends S at the join and switches to its loop,

| *worker A* | *worker B* |
|---|---|
| (1) push(&t); $f_1$() on S | steal t from A |
| | fresh stack T; $f_2$() on T |
| (2) pop() returns **null** | |
| (4) swap(j.k, A.k) | |
| in loop: j.pending = 1 | |
| steal other work ... | |
| | $f_2$() returns; finish(&j) |
| | in loop: j.pending = 0 |
| | swap(B.k, j.k): the swap |
| | at (4) returns, on S |

**Figure 3.** The second branch is stolen by worker B, which reaches the join last

which brings the count to 1. A then looks for other work. When $f_2$ returns, B switches to its loop, which brings the count to 0. So B resumes the context saved in the join: the call to swap at step 4 returns, on stack S as before, but now on worker B, and fork2join returns there. Had A reached the join last, A would have found the count at 1, brought it to 0, and resumed the parent itself.

So a call to fork2join can begin on one thread and return on another. The stack stays the same throughout; only the thread that runs it changes.

## 4 Why it is safe

The correctness of the algorithm rests on four facts.

***A branch is counted only after it has left its stack.*** As soon as the count of a join reaches zero, another worker may resume the parent on stack S, and the parent may return from fork2join, which pops the frame that holds the join and the task, and frees the stack of the stolen task. So the first branch must be off S, and the second off its stack, T, before either is counted. This is why a branch switches to the scheduler loop to be counted, instead of decrementing in place, and why the loop does not touch the join after its decrement, unless it brought the count to zero.

***No two workers run on the same stack.*** A suspended parent is reachable only through the context saved in its join. Each branch decrements the count once, so exactly one worker brings it to zero, and only that worker resumes the context. By then, the context has been saved: the first branch is counted after the swap at step 4 has saved it.

***The pop at step 2 is correct even if*** $f_1$ ***moved.*** $f_1$ returns on a different worker only if a task pushed inside $f_1$ was stolen. Thieves take the oldest task of a deque, and the task of this fork is older than any task pushed inside $f_1$. So by then it has been stolen as well. And the worker on which $f_1$ returns resumed this stack from its loop, with an empty deque, so the pop there correctly fails.

***Only tasks that have never run are stolen.*** A deque holds only tasks pushed at step 1, which have not started. A suspended parent is never in a deque, so no thief can resume it.

## 5 Costs and consequences

***Costs.*** A fork whose second branch is not stolen makes one push and one pop, looks up its worker twice, and calls $f_1$ and $f_2$. It switches contexts not at all, allocates nothing, because the join and the task live in the frame of fork2join, and performs no atomic operation beyond those of the deque. With micro taskparts, such a fork takes about 6 ns on an Apple M5 (Appendix D). A fork whose second branch is stolen allocates one fresh stack, of a fixed size, which is touched only as deep as $f_2$ recurses, and is freed when the parent returns from fork2join. It also makes four context switches: to the stolen task and back to the thief's loop, and from the parent to a loop and back to the parent. Appendix E bounds the number of stacks in use.

***Consequences for client code.*** Code that follows a call to fork2join may run on a different thread from the code that precedes it. So client code must not hold a lock across fork2join, nor expect a thread-local variable or the thread id to be the same afterward. More subtly, compilers assume that a function never changes threads, and so may reuse a thread-local address computed before the call. Then even a fresh read of a thread-local variable after fork2join, in the same function, can return the old thread's copy. Reading thread-local state through a function that the compiler can neither inline nor treat as pure avoids this problem. Other consequences follow from the use of task stacks. Task stacks have a fixed size, and no guard page by default. Exceptions must not escape $f_1$ or $f_2$, because unwinding cannot cross from a task stack into the scheduler. Backtraces end at the bottom of a task stack. And sanitizers, such as AddressSanitizer and ThreadSanitizer, are not told about the stack switches, so they may report false errors.

***Implementations.*** Micro taskparts implements the algorithm of Figures 1 and 2, with a lock-free deque [2], and nothing else, in about 500 lines of C++, including the assembly for x86-64 and AArch64. It is in the directory doc/native-fork-join/microtaskparts of the TaskPaRTS repository [14], and numbers the steps of fork2join as in Figure 2. TaskPaRTS implements an earlier formulation of native fork join, in which every fork passes through the scheduler loop (Appendix C).

## References

[1] Arm Limited. Procedure call standard for the Arm 64-bit architecture (AArch64). https://github.com/ARM-software/abi-aa.

[2] N. S. Arora, R. D. Blumofe, and C. G. Plaxton. Thread scheduling for multiprogrammed multiprocessors. In *SPAA '98*, pages 119–129, 1998.

[3] G. E. Blelloch, D. Anderson, and L. Dhulipala. ParlayLib – a toolkit for parallel algorithms on shared-memory multicore machines. In *SPAA '20*, pages 507–509, 2020.

[4] R. D. Blumofe and C. E. Leiserson. Scheduling multithreaded computations by work stealing. *J. ACM*, 46:720–748, Sept. 1999.

[5] D. Chase and Y. Lev. Dynamic circular work-stealing deque. In *SPAA '05*, pages 21–28, 2005.

[6] K.-F. Faxén. Wool—a work stealing library. *SIGARCH Computer Architecture News*, 36(5):93–100, 2008.

[7] M. Frigo, C. E. Leiserson, and K. H. Randall. The implementation of the Cilk-5 multithreaded language. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation*, PLDI '98, pages 212–223. ACM, 1998.

[8] S. C. Goldstein, K. E. Schauser, and D. E. Culler. Lazy threads: Implementing a fast parallel call. *Journal of Parallel and Distributed Computing*, 37(1):5–20, 1996.

[9] R. H. Halstead, Jr. Implementation of Multilisp: Lisp on a Multiprocessor. In *Proceedings of the 1984 ACM Symposium on LISP and functional programming*, LFP '84, pages 9–17. ACM, 1984.

[10] Intel. Intel threading building blocks, 2011. https://www.threadingbuildingblocks.org/.

[11] I.-T. A. Lee, S. Boyd-Wickizer, Z. Huang, and C. E. Leiserson. Using memory mapping to support cactus stacks in work-stealing runtime systems. In *Proceedings of the 19th International Conference on Parallel Architectures and Compilation Techniques*, PACT '10, pages 411–420, New York, NY, USA, 2010. ACM.

[12] H. J. Lu, M. Matz, M. Girkar, J. Hubička, A. Jaeger, and M. Mitchell. System V application binary interface: AMD64 architecture processor supplement. https://gitlab.com/x86-psABIs/x86-64-ABI.

[13] E. Mohr, D. A. Kranz, and R. H. Halstead, Jr. Lazy task creation: A technique for increasing the granularity of parallel programs. *IEEE Transactions on Parallel and Distributed Systems*, 2(3):264–280, 1991.

[14] M. Rainey. TaskPaRTS: a task-parallel run-time system for C++. https://github.com/mikerainey/taskparts.

[15] M. Rainey. Running time of `fib(38)` with seven builds of micro taskparts. Measurements certified by Flexibench 2a10c0c, certificate `a5cf1c2f58147214740c9af4ea1a575e5a9f08cb18efc5bd0dc1ad28cd5b684d`, Nov. 2026. In the directory `doc/native-fork-join/perf` of the TaskPaRTS repository, https://github.com/mikerainey/taskparts, whose README lists the commit of each build and how to check the certificate.

[16] A. Robison. A primer on scheduling fork-join parallelism with work stealing. Technical Report N3872, ISO/IEC JTC1/SC22/WG21, 2014.

[17] T. B. Schardl, W. S. Moses, and C. E. Leiserson. Tapir: Embedding fork-join parallelism into llvm's intermediate representation. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 249–265, 2017.

[18] F. Schmaus, N. Pfeiffer, T. Hönig, J. Nolte, and W. Schröder-Preikschat. Nowa: A wait-free continuation-stealing concurrency platform. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 360–371, 2021.

[19] S. Seo, A. Amer, P. Balaji, C. Bordage, G. Bosilca, A. Brooks, P. Carns, A. Castelló, D. Genet, T. Herault, S. Iwasaki, P. Jindal, L. V. Kalé, S. Krishnamoorthy, J. Lifflander, H. Lu, E. Meneses, M. Snir, Y. Sun, K. Taura, and P. Beckman. Argobots: A lightweight low-level threading and tasking framework. *IEEE Transactions on Parallel and Distributed Systems*, 29(3):512–526, 2018.

[20] K. Taura, K. Tabata, and A. Yonezawa. StackThreads/MP: Integrating futures into calling standards. In *Proceedings of the 7th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP '99, pages 60–71. ACM, 1999.

[21] D. B. Wagner and B. G. Calder. Leapfrogging: A portable technique for implementing efficient futures. In *Proceedings of the Fourth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP '93, pages 208–217. ACM, 1993.

[22] C. J. Williams and J. Elliott. Libfork: Portable continuation-stealing with stackless coroutines. *IEEE Transactions on Parallel and Distributed Systems*, 36(5):877–888, 2025.

[23] C. Yang and J. Mellor-Crummey. A practical solution to the cactus stack problem. In *Proceedings of the 28th ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '16, pages 61–70. ACM, 2016.

```
swap:
  movq %rbx, 0(%rdi)
  movq %rbp, 8(%rdi)
  movq %r12, 16(%rdi)
  movq %r13, 24(%rdi)
  movq %r14, 32(%rdi)
  movq %r15, 40(%rdi)
  leaq 8(%rsp), %rdx
  movq %rdx, 48(%rdi)
  movq (%rsp), %rdx
  movq %rdx, 56(%rdi)
  movq 0(%rsi), %rbx
  movq 8(%rsi), %rbp
  movq 16(%rsi), %r12
  movq 24(%rsi), %r13
  movq 32(%rsi), %r14
  movq 40(%rsi), %r15
  movq 48(%rsi), %rsp
  jmpq *56(%rsi)
```

```
restore:
  movq 0(%rdi), %rbx
  movq 8(%rdi), %rbp
  movq 16(%rdi), %r12
  movq 24(%rdi), %r13
  movq 32(%rdi), %r14
  movq 40(%rdi), %r15
  movq 48(%rdi), %rsp
  jmpq *56(%rdi)
```

**Figure 4.** Switching contexts on x86-64

## A Machine contexts

A context holds the registers that the calling convention requires a callee to preserve [1, 12], plus the stack pointer and the address at which to resume:

| | x86-64 | AArch64 |
|---|---|---|
| callee-saved | `rbx`, `rbp`, `r12`–`r15` | `x19`–`x29`, `d8`–`d15` |
| stack pointer | `rsp` | `sp` |
| resume address | return address | `lr` (`x30`) |
| total | 8 words | 21 words |

Caller-saved registers need no saving: to the compiler, `swap` is an ordinary call, so the compiler has already spilled any live caller-saved registers before calling it. Figure 4 shows both functions for x86-64. `swap` first saves into $k_1$, which it receives in `%rdi`. It records, as the stack pointer to resume with, the value the stack pointer will have once `swap` has returned, and, as the resume address, its own return address. It then reloads the registers from $k_2$, in `%rsi`, switches stacks, and jumps to the resume address. `restore` is the second half alone. The AArch64 versions are analogous. They too resume with an indirect branch to the restored `lr`, `br x30`, rather than with `ret` (Appendix D).

***Fresh contexts.*** To start a function $f$ on a fresh stack $s$, the scheduler makes a context with `new_context(`$s$`, `$f$`)`. The stack pointer of this context is the top of $s$, aligned to 16 bytes, its resume address is $f$, and its other registers are zero. On x86-64, the stack pointer is a further 8 bytes down, as if a `call` had pushed a return address. Resuming the context then starts $f$ on $s$, as if $f$ had been called there.

## B The deque

Any work-stealing deque will do, as long as thieves take the oldest task, which Section 4 relies on. The owner pushes and pops at one end, and thieves take from the other. Micro taskparts uses the lock-free deque of Arora et al. [2]; the deque of Chase and Lev [5] would serve as well. A deque holds at most one task per call to `fork2join` in progress on the stack that its worker is running, so a small, fixed capacity suffices.

## C Fork join through the scheduler

TaskPaRTS implements an earlier formulation of native fork join, in which every fork passes through the scheduler loop, whether or not anything is stolen. We describe it here because the algorithm of Section 2 was distilled from it, and because Appendix D measures both.

In this formulation, the scheduler loop is the only place where a task starts, resumes, or finishes. Each task has a count of the tasks it is waiting for, a pointer to the task that is waiting for it, a context, and an action, which tells the loop what to do with the task: start it on a fresh stack, resume its context, or, the next time it yields, report to the task that is waiting for it that it has finished. The loop pops or steals a task, and starts or resumes it. When a task yields as finished, the loop decrements the count of the task that is waiting for it, and makes that task ready if the count reaches zero.

The parent of a fork is the task that the worker is currently running. `fork2join` creates two tasks that the parent waits for: one for the second branch, and one that stands for the parent while it runs the first branch inline. It saves the parent's context before anything else, with a function that, like `setjmp`, returns a second time when the context is resumed. It then yields to the scheduler loop three times, even when nothing is stolen: to start the first branch, to finish the first branch and start the second, and to finish the second branch and continue as the parent. Each yield is a round trip, so a fork makes six context switches. A task is made ready before its context is saved, so the formulation also needs a private slot in front of each deque, which keeps the task made ready most recently out of the reach of thieves.

The algorithm of Section 2 is what remains once the round trips are removed. The count and the context of the parent become the join, the task for the second branch remains, and the stand-in task, the actions, the private slot, and the second return disappear. What the two share is the technique: a parent whose second branch was stolen is suspended on its own stack, a branch is counted from the scheduler loop, and the worker that counts last resumes the parent. What the formulation of this appendix offers in exchange for its cost is uniformity: every transition of every task passes through one loop, which is a convenient place for a scheduler to keep statistics, log events, or put workers to sleep.

| version | workers 1 | 4 | 8 |
|---|---|---|---|
| *through the scheduler* (Appendix C) | | | |
| deque protected by a lock | 5.64 | 1.46 | 1.23 |
| lock-free deque | 4.99 | 2.72 | 3.21 |
| + victims chosen by hashing | 4.96 | 1.31 | 0.88 |
| + one cache line per task field | 4.92 | 1.30 | 0.88 |
| + fewer lookups of the worker | 4.53 | 1.18 | 0.75 |
| + context switch in one call | 1.86 | 0.49 | 0.33 |
| *directly* (Section 2) | | | |
| no round trips | 0.36 | 0.10 | 0.07 |

**Table 1.** Running time of `fib(38)`, in seconds, with micro taskparts, as each change is applied in turn: the median of five runs

## D Performance notes

A fork whose second branch is not stolen does little work, so small choices in the implementation show up in the running time. This appendix records the choices that we measured with seven versions of micro taskparts, running `fib(38)`, which makes 63 million forks, on an Apple M5 with 4 performance and 6 efficiency cores (Table 1). The first six versions implement the formulation of Appendix C, and the last implements the algorithm of Section 2. Each time in the table is the median of five runs, and the runs of the seven versions were interleaved. Flexibench recorded the runs, checked that every version computes correct results, and certified the values in the table [15]. The record is in the directory `doc/native-fork-join/perf` of the TaskPaRTS repository [14].

***No round trips.*** The choice that matters most is the one between the two formulations. Without the round trips through the scheduler loop, `fib(38)` takes 0.36 s on one worker, instead of 1.86 s with the fastest version that makes them: about 6 ns per fork instead of 29. The remaining paragraphs concern the versions that make the round trips, in the order in which we arrived at them. Of their lessons, the choice of deque, the choice of victims, and the way contexts are switched carry over to the last version, which makes all three choices as the sixth version does.

***A lock-free deque.*** The first version protects each deque with a lock. This is the simplest correct design, but a naive one: it has long been known that the owner of a deque need not, and should not, take a lock in the common case [2, 5, 7]. We include it as a baseline, to show what synchronizing on the deque costs on this machine. With the lock, a fork takes two uncontended locks, one to schedule the second branch and one to pop it. The lock-free deque of Appendix B removes them, and is about 10% faster on one worker. So, for `fib`, the deque was not the bottleneck: most of the time of a fork went elsewhere, as the later rows of the table show. But the lock had hidden the next problem.

***A slowdown with several workers.*** The second version in the table scales poorly, and we have not found out why. On 4 workers, it takes 2.72 s, about twice as long as the third version (1.31 s), and on 8 workers, it takes longer still (3.21 s). The two versions differ only in how a worker chooses a victim to steal from. In the second version, each worker draws from a random-number generator of its own, of type `std::minstd_rand`. In the third, it hashes its id and a count of its steal attempts. This difference should not matter, because steals are rare: workers attempt only a few hundred per run. Nor does the slowdown need workers to keep stealing from one another: it persisted even when we allowed a single steal, after which the two workers ran disjoint subcomputations.

In earlier measurements, outside the table, three other changes each removed the slowdown, with the generator still in place.

- Starting the count, the pointer, and the context of a task each on a cache line of its own.
- Updating the count with plain loads and stores, rather than with an atomic decrement, when the second branch is not stolen.
- Adding any atomic operation just before the pop that follows $\texttt{f}_1$, even one with no memory ordering. The lock of the first version is such an operation, which is how it had hidden the slowdown.

These changes have little in common with one another, or with the choice of victim. So the slowdown seems to depend on details of the code and of the layout of data, rather than on the algorithm, and it may reappear after an unrelated change. As a precaution, the fourth version adds the first of these changes, although, once victims are chosen by hashing, it makes no measurable difference.

***Fewer lookups of the worker.*** Each use of `me` looks up the worker in thread-local storage, through a function that the compiler can neither inline nor treat as pure (Section 5). The compiler barrier that enforces this emits no instruction. The cost is the call and the lookup, which on macOS is itself a call. But in a fork, the worker changes only across the calls to $\texttt{f}_1$ and $\texttt{f}_2$, and across the second return of the function that saved the parent's context. The three yields to the scheduler resume the task on the same worker, because the task is in that worker's private slot. And the scheduler loop never changes worker. Looking up the worker once per stretch, rather than at each use, cuts the lookups per fork from about 12 to 3, and the running time on one worker by about 8%. The algorithm of Section 2 needs two lookups per fork, one before the push and one after $\texttt{f}_1$.

***A context switch in one call.*** In the first five versions, a context switch is two calls, one that saves a context and then `restore`, and, on AArch64, `restore` resumes with `ret`.

The processor predicts where a `ret` goes from a stack of the return addresses of recent calls, and after a context switch, this prediction is wrong. The sixth version switches with the single routine `swap` of Appendix A, and resumes with an indirect branch, `br x30`, instead. This makes `fib(38)` more than twice as fast on 1, 4, and 8 workers. On one worker, it takes 1.86 s instead of 4.53 s, or about 29 ns per fork instead of 72. In measurements outside the table, each half of the change helped on its own, the single routine more than the indirect branch, but we have not established why the single routine helps as much as it does. On x86-64, `restore` already resumes with an indirect jump (Figure 4), and we have not measured the single routine there.

| strategy | recip. | time | space |
|---|---|---|---|
| 1. Recompile everything | no | very strong | very strong |
| 2. One stack per worker | yes | very strong | no |
| 3. Depth-restricted stealing | yes | no | very strong |
| 4. Limited-depth stacks | yes | no | very strong |
| 5. New stack when needed | yes | very strong | weak |
| 6. Recycle ancestor stacks | yes | strong | weak |
| 7. TLMM cactus stacks | yes | strong | strong |
| Native fork join | yes | very strong[†] | weak |

**Table 2.** Strategies for managing stacks under work stealing, from Lee et al. [11], with native fork join added: whether each provides serial-parallel reciprocity, and how strong its time and space bounds are. [†]Expected, but not proved.

## E Native fork join and the cactus-stack problem

This appendix places native fork join among other ways of managing stacks under work stealing. It starts from the survey of Lee et al. [11], and ends with systems outside that survey.

***The cactus-stack problem.*** Lee et al. ask for three properties at once. The first is *serial-parallel reciprocity*: parallel code and serial code, including legacy binaries compiled for an ordinary linear stack, can call each other freely. The second is a time bound $T_P \le T_1/P + c_\infty T_\infty$, where $T_1$ is the work, $T_\infty$ the span, and $P$ the number of workers. The third is a space bound $S_P \le PS_1$, where $S_1$ is the stack space of a serial execution. Meeting all three is what they call the *cactus-stack problem*. They survey seven strategies, which Table 2 summarizes.

Strategy 1, used by Cilk-5, allocates every frame on the heap, which requires recompiling all code, serial code included. Strategies 2 to 6 keep linear stacks. They differ in what a worker does when it must steal while its stack holds a frame that it cannot pop, because a thief is running the continuation of that frame. The worker may steal onto the same stack anyway (2), steal only frames deeper than those on its stack (3), stop stealing while its stack is too deep (4), take a new stack (5), or take the stack of an ancestor of the stolen frame, if that ancestor is suspended with no frame above it (6). Strategy 7, Cilk-M, maps the frames that a thief shares with its victim into the stack of each, at the same virtual addresses, which requires support from the operating system.

***Where native fork join fits.*** Native fork join is not orthogonal to these strategies: it manages stacks in the same way as Strategy 5. A thief always runs a stolen task on a fresh stack. A worker that reaches a join whose second branch is still running leaves its stack suspended and returns to its scheduler, so whatever it steals next runs on another fresh stack. And a stack is freed only once it holds no frame that can still run. What is new is how native fork join gets there: by stealing children rather than continuations, and by switching contexts, which needs no support from the compiler or the operating system.

***Children, not continuations.*** Lee et al. assume continuation stealing, as in Cilk, and set child stealing aside, because it can need unbounded bookkeeping, even on one processor. That happens when a parent can keep forking before it runs any child, as in a loop of spawns. Binary fork join rules this out: a fork pushes one task and runs $\texttt{f}_1$ at once, so a worker's deque holds at most one task per call to `fork2join` in progress on its stack, just as a Cilk deque holds at most one frame per spawn in progress. In fact, for binary fork join, the two disciplines steal the same work. $\texttt{fork2join(f}_1\texttt{, f}_2\texttt{)}$ corresponds to spawning $\texttt{f}_1$, calling $\texttt{f}_2$, and syncing, and the stolen task corresponds to the continuation of the spawn, up to the sync. In both, whoever reaches the join last runs the code that follows it, which is why that code can run on a different thread from the code that precedes the fork.

***No frame spans two stacks.*** The difference that matters is how a computation moves from one worker to another. Under continuation stealing, a thief resumes the stolen frame in place. In Strategy 2, for example, the thief points its base pointer at the frame, which stays on the victim's stack, and its stack pointer at its own stack, where the frame's callees then run. So parallel code must be compiled to work with a frame and its callees on different stacks, or, in Cilk-M, separated by a gap. Lee et al. list seven invariants that compiled Cilk-M functions must maintain, and hand-compiled their benchmarks to meet them.

Native fork join never splits a frame between stacks. A thief runs only the thunk $\texttt{f}_2$, as an ordinary call on its own stack, and the parent resumes on its own stack, by restoring a context saved by an ordinary call to `swap`. The only pointers from one stack into another are those that client code creates, such as the references that $\texttt{f}_2$ captures, which are ordinary C++. So native fork join provides Strategy 5 in a library, with an unmodified compiler and operating system. Serial-parallel

reciprocity holds in the sense of Lee et al.: `fork2join` is an ordinary function, which any serial code can call, including a legacy binary that calls back into client code. Two caveats from Section 5 fall outside their criteria. The code after a join may run on another thread, which is also true under continuation stealing. And the frames on a task stack must fit in its fixed size.

***Time.*** Under native fork join, no worker ever waits at a join, and a steal costs one stack allocation. Moreover, execution fits the DAG model analyzed by Arora et al. [2]. A fork pushes one of its branches and runs the other next, and a parent that a join makes ready runs next on the worker that made it ready. So we expect native fork join to meet the time bound of Strategy 5, with a constant $c_\infty$, but we have not written out a proof. By contrast, library schedulers that help while waiting, such as those of TBB and ParlayLib (Section 1), keep one stack per worker, as Strategies 2 to 4 do. Lee et al. liken the heuristic of TBB to Strategy 3, which can lose the time bound.

***Space.*** Native fork join pays the price of Strategy 5. As Lee et al. note for Strategy 5, the busy-leaves property holds: a stack suspended at a join waits on a stolen task, whose stack is either running or itself suspended at a join, and so on, down to a stack that a worker is running. So the stacks in use form a tree with at most $P$ leaves. Each stack starts at a task nested more deeply in calls to `fork2join` than the task that starts its parent, so the tree has depth at most $D$, the maximum nesting depth of calls to `fork2join`, and there are $O(PD)$ stacks in use. This count holds even if the stack of a stolen task counts as in use until its parent returns from `fork2join` (Section 4). Lee et al. bound the physical memory that Strategy 5 uses for frames by $PS_1$. But a stack can grow as large as $S_1$, and is recycled only when it holds no frames, so stacks can take up to $DPS_1$ of virtual memory. In native fork join, each stack has a fixed size $Z$, a parameter of the implementation, so stacks take $O(PDZ)$ of virtual memory. The operating system gives a page of a stack physical memory only when the page is first touched, so a generous $Z$ costs address space rather than memory. But a page of a stack, once touched, stays in memory at least until the stack is freed.

***Refinements.*** Lee et al. mention two refinements of Strategy 5 that appear to carry over to native fork join, and that micro taskparts does not implement. One is to unmap the pages of frames once they are popped, to give their physical memory back, which they expect to be costly unless done lazily. The other is Strategy 6, in which a thief runs a stolen task on the stack of a suspended ancestor, when no frame lies above that ancestor's frame. This is safe, because the ancestor cannot resume before the stolen task finishes. But Lee et al. report that Strategy 6 can still need $\Omega(P^2S_1)$ stack space.

***Paying only when stolen.*** The idea that a fork should cost little more than a call unless its work is actually stolen goes back to lazy task creation [13] and to lazy threads [8]. Lazy task creation steals the continuation of the parent, which requires splitting a stack, and is built into a compiler. Lazy threads, like native fork join, run a child on the parent's stack as if it had been called sequentially, and give a child a stack of its own only when another processor asks for work. But lazy threads rely extensively on compiler support, whereas native fork join uses ordinary calls and the standard calling convention. StackThreads/MP [20] also keeps the standard calling convention, with modest extensions to the compiler. It migrates the parent, and leaves the frames of suspended threads in place within the worker's stack.

***Continuation stealing in a library.*** Fibril [23] is the system closest to native fork join in its treatment of stacks. It is a C library that needs no custom compiler, and, like Strategy 5, it runs stolen work on a stack taken from a pool. It steals continuations: a thief resumes the stolen frame where it lies, on the victim's stack, with a stack pointer that points into a new stack, which requires every such function to keep a frame pointer. A worker that must wait at a join unmaps the unused part of its stack, which gives Fibril a strong space bound, $S_P \leq P(S_1 + D)$, where native fork join has only the weak one. Nowa [18] builds on Fibril's stacks, and makes the coordination of joins wait-free. Libfork [22] obtains continuation stealing portably, from the stackless coroutines of C++20, at the price that tasks are written as coroutines, whose frames it allocates on segmented stacks. Native fork join steals children instead, so no frame is ever split between stacks, and client code consists of ordinary functions.

***Child stealing that does not wait.*** Robison [16] classifies work-stealing schedulers by what is stolen, the child or the continuation, and by what happens at a join that must wait: the worker *stalls*, or the scheduler is *greedy*, and the last worker to reach the join continues past it. The child-stealing libraries in his table, among them TBB, stall, and the only examples of greedy child stealing are an unpublished scheduler and the untied tasks of OpenMP. Native fork join belongs to this last, sparsely populated class. Wool [6] has the same fast path as native fork join: a spawned task that is not stolen is run by an ordinary call. But when the task was stolen and has not finished, the worker stays at the join and steals back from the thief, a strategy called leapfrogging [21], which bounds the depth of the worker's stack by restricting what the worker may steal. Native fork join places no such restriction: the worker is free to steal anything, because what it steals runs on another stack.

***Stacks per steal, not per thread.*** A library of user-level threads typically gives each thread a stack when the thread is created. In Argobots [19], for example, each user-level

thread has its own, dynamically allocated stack. Native fork join creates a stack only when a task is stolen; a task that is not stolen runs on the stack of its parent. So a computation allocates one stack per steal, rather than one per fork, and work stealing makes $O(PT_\infty)$ steal attempts in expectation [2, 4].